

Relatório 8 - Prática: Criação do Primeiro Dataset Sintético (III)

Ana Clara Fortunato de Souza

Descrição da atividade

1. Introdução

A atividade teve como base os dois primeiros vídeos do curso 3D Rendered Datasets in Blender for Beginners, que apresentam um processo introdutório para a criação de datasets sintéticos utilizando o Blender.

O primeiro vídeo concentra-se na preparação da cena tridimensional. São abordados os principais elementos necessários para produzir imagens adequadas, como o plano de fundo, os objetos, os materiais, a iluminação, a câmera e as configurações de renderização.

O segundo vídeo apresenta a automação desse processo por meio de scripts em Python. Utilizando a API do Blender, é possível modificar propriedades dos objetos, como rotação e cor, além de executar diversas renderizações automaticamente.

A combinação dessas duas etapas transforma uma cena criada manualmente em um sistema programável de geração de dados. Dessa forma, uma mesma estrutura tridimensional pode produzir diferentes imagens, aumentando a variedade do conjunto sem a necessidade de modificar cada amostra manualmente.

[Link para o repositório com o código do vídeo.](#)

2. Preparação da cena no Blender

A primeira parte do curso apresenta os conhecimentos fundamentais para configurar uma cena destinada à geração de imagens sintéticas.

O exemplo demonstrado utiliza objetos tridimensionais simples, especialmente as letras A, B e C. A utilização de objetos facilmente identificáveis facilita a compreensão do processo, pois permite observar claramente as alterações produzidas pela rotação, pelos materiais e pela iluminação.

2.1 Criação do plano de fundo

O plano de fundo funciona como uma espécie de pequeno estúdio para a renderização. Ele é formado por superfícies posicionadas ao redor do objeto, permitindo que o cenário ocupe completamente o campo de visão da câmera.

Essa configuração evita que áreas vazias ou elementos indesejados apareçam nas imagens. Além disso, o fundo interfere diretamente na iluminação, nas sombras e no contraste entre os objetos e o ambiente.

Uma cena destinada à geração de dados precisa apresentar uma composição visual consistente. Caso o plano de fundo seja muito claro, muito escuro ou possua elementos desnecessários, o objeto principal poderá ficar difícil de identificar.

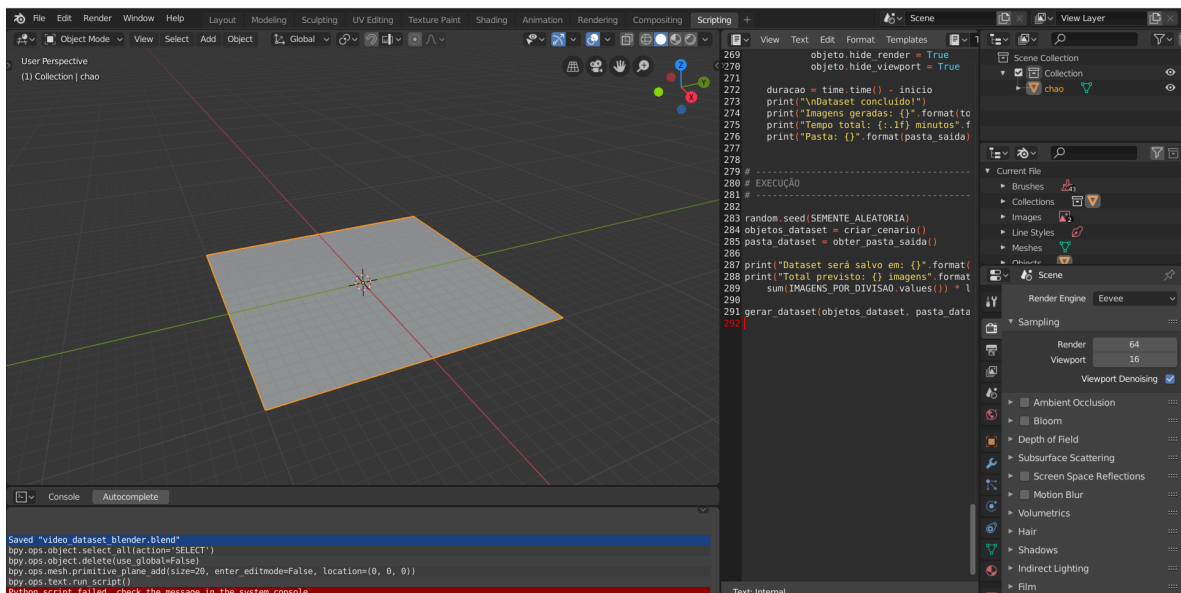


Figura 1 – Configuração da cena utilizada para a geração das imagens sintéticas.

Fonte: elaboração própria.

2.2 Modelagem e organização dos objetos

Os objetos utilizados no curso são criados como textos tridimensionais. Depois da inserção das letras, são aplicados ajustes de tamanho, posicionamento, orientação e profundidade.

A extrusão adiciona volume às letras, transformando elementos originalmente planos em objetos tridimensionais. Esse volume permite que as diferentes rotações revelem faces, sombras e silhuetas variadas.

Outro aspecto importante é a nomenclatura dos objetos. Os nomes definidos no Blender são posteriormente utilizados pelo código Python para localizar e modificar cada elemento da cena. Por esse motivo, nomes claros e padronizados reduzem erros durante a automação.

Por exemplo, um script pode buscar diretamente um objeto chamado “A”. Caso o objeto tenha sido renomeado ou possua outro nome dentro da cena, o código não conseguirá acessá-lo corretamente.

2.3 Materiais

Os materiais determinam a aparência visual dos objetos. No curso, eles são configurados para permitir que suas cores sejam modificadas posteriormente por meio do script.

O material utiliza o sistema de nós do Blender, especialmente o nó Principled BSDF, que reúne propriedades relacionadas à cor, brilho, rugosidade, reflexão e aparência da superfície.

Para que a mudança automática de cor funcione, o material deve estar corretamente associado ao objeto e com o sistema de nós habilitado. Dessa forma, o Python consegue acessar o parâmetro Base Color e atribuir uma nova cor antes de cada renderização.

A alteração dos materiais é uma estratégia importante para aumentar a diversidade do dataset. Um mesmo objeto pode aparecer com várias cores, reduzindo a possibilidade de o modelo associar uma classe exclusivamente a uma característica visual específica.

2.4 Iluminação

A iluminação influencia diretamente a aparência das imagens. A posição, a intensidade e o tipo das fontes de luz determinam a formação das sombras, o contraste e a visualização do volume dos objetos.

No contexto de dados sintéticos, a luz não representa apenas uma escolha estética. Ela também simula diferentes condições de aquisição das imagens.

Uma iluminação inadequada pode causar problemas, como:

- objetos muito escuros;
- perda de detalhes;
- reflexos excessivos;
- sombras intensas;
- pouco contraste com o fundo.

Por isso, antes de iniciar a geração automática, é importante realizar renderizações de teste e verificar se os objetos permanecem reconhecíveis em diferentes orientações.

2.5 Configuração da câmera

A câmera determina o enquadramento e o ponto de vista das imagens. No curso, ela é posicionada de forma que os objetos permaneçam visíveis mesmo após as alterações de rotação.

Esse cuidado é necessário porque um objeto pode ocupar uma área maior ou menor da imagem dependendo de sua orientação. Caso a câmera esteja muito próxima, algumas partes podem ser cortadas. Caso esteja muito distante, o objeto pode ficar pequeno e perder detalhes importantes.

A câmera também precisa ser definida como a câmera ativa da cena. Caso essa configuração não seja realizada corretamente, o Blender poderá utilizar outra câmera ou impedir a execução esperada da renderização.

3. Configuração da renderização

Após a preparação da cena, são definidas as propriedades utilizadas para produzir as imagens.

Entre as principais configurações estão:

- motor de renderização;
- resolução da imagem;
- formato do arquivo;
- qualidade;
- transparência do fundo;
- diretório de saída;
- câmera ativa.

O curso utiliza imagens em formato PNG, que oferece boa qualidade e é amplamente utilizado em projetos de visão computacional.

A resolução pode ser reduzida durante os testes iniciais, diminuindo o tempo necessário para cada renderização. Depois da validação do código, uma resolução maior pode ser utilizada na geração final.

Antes de iniciar centenas de renderizações, é recomendável produzir algumas imagens de teste. Esse procedimento permite verificar o enquadramento, as cores, a iluminação, a visibilidade dos objetos e os caminhos dos arquivos.

Caso exista algum erro na cena-base, o processo automatizado apenas repetirá esse erro várias vezes — uma eficiência admirável, mas na direção errada.

4. Automação com Python

A segunda parte do curso apresenta o ambiente de programação do Blender. Por meio do workspace Scripting, é possível criar, executar e testar scripts diretamente no programa.

O principal módulo utilizado é o `bpy`, responsável por disponibilizar acesso aos objetos, materiais, luzes, câmeras e configurações da cena.

Um objeto pode ser localizado por meio do seguinte comando:

```
import bpy
```

```
objeto = bpy.data.objects["A"]
```

Após acessar o objeto, o programa pode modificar suas propriedades, como posição, escala, rotação e visibilidade.

A utilização do Python permite transformar uma tarefa manual e repetitiva em um processo automatizado. Em vez de alterar a cena, clicar no botão de renderização e salvar cada imagem individualmente, o script realiza essas etapas em sequência.

5. Randomização da rotação

Uma das primeiras técnicas apresentadas é a alteração aleatória da rotação dos objetos.

No Blender, os ângulos são representados internamente em radianos. Uma rotação completa corresponde a 2π radianos. Dessa forma, valores aleatórios podem ser multiplicados por 2π para gerar rotações entre 0° e 360° .

Um exemplo simplificado é apresentado abaixo:

```
import bpy
```

```
import random
```

```
import math
```

```
objeto = bpy.data.objects["A"]
```

```
objeto.rotation_euler[0] = random.random() * 2 * math.pi
```

```
objeto.rotation_euler[1] = random.random() * 2 * math.pi
```

```
objeto.rotation_euler[2] = random.random() * 2 * math.pi
```

Nesse código, os eixos X, Y e Z recebem valores aleatórios. Como resultado, o objeto pode ser apresentado em diferentes orientações a cada execução.

Essa variação ajuda a criar um conjunto mais diversificado. Entretanto, os valores precisam ser avaliados para evitar imagens em que o objeto fique irreconhecível, muito inclinado ou parcialmente escondido.

6. Randomização das cores

Além da rotação, o curso demonstra a alteração automática das cores dos objetos.

O material pode ser acessado pelo script e seu parâmetro de cor pode ser modificado antes de cada renderização. A utilização do sistema HSV facilita a geração de cores diferentes, mantendo valores adequados de saturação e brilho.

No sistema HSV:

- **Hue** representa o matiz ou a tonalidade;
- **Saturation** representa a intensidade da cor;
- **Value** representa o brilho.

Ao alterar principalmente o matiz e manter saturação e brilho relativamente altos, são produzidas cores vivas e visualmente distintas.

Um exemplo simplificado seria:

```
import random
```

```
import colorsys
```

```
h = random.random()
```

```
s = 0.8
```

```
v = 0.9
```

```
r, g, b = colorsys.hsv_to_rgb(h, s, v)
```

Os valores obtidos podem ser aplicados ao parâmetro **Base Color** do material.

A randomização das cores aumenta a diversidade do dataset e impede que determinada classe fique associada exclusivamente a uma cor.

7. Controle de visibilidade

Como a cena possui mais de uma classe, é necessário controlar quais objetos aparecem em cada renderização.

Durante a geração das imagens de uma determinada letra, os outros objetos podem ser ocultados. Assim, apenas a classe selecionada permanece visível para a câmera.

Esse controle pode ser realizado por propriedades como:

```
objeto.hide_render = True
```

Quando `hide_render` recebe o valor `True`, o objeto permanece na cena, mas não aparece na imagem renderizada.

O script pode inicialmente ocultar todas as classes e, em seguida, tornar visível somente o objeto correspondente à imagem atual.

Essa estratégia garante que os arquivos sejam armazenados corretamente nas pastas de cada classe e evita a presença accidental de mais de um objeto na mesma imagem.

8. Utilização de laços de repetição

Os laços de repetição são responsáveis pela escalabilidade do processo.

Em cada repetição, o programa pode:

1. selecionar uma classe;
2. tornar o objeto correspondente visível;
3. sortear uma nova rotação;
4. escolher uma nova cor;
5. definir o nome e o caminho do arquivo;
6. renderizar a cena;
7. salvar a imagem;
8. repetir o procedimento.

Uma estrutura simplificada pode ser representada da seguinte maneira:

```
for i in range(10):
```

```
    randomizar_rotacao()
```

```
    randomizar_cor()
```

```
        bpy.context.scene.render.filepath = f"imagem_{i}.png"
```

```
        bpy.ops.render.render(write_still=True)
```

O parâmetro `write_still=True` faz com que o resultado da renderização seja gravado automaticamente no caminho definido.

A partir desse processo, dezenas, centenas ou milhares de imagens podem ser produzidas sem intervenção manual durante cada renderização.

9. Organização do dataset

Um dataset utilizado em aprendizado de máquina geralmente é dividido em três subconjuntos:

Divisão	Finalidade
Treino	Utilizado para ajustar os parâmetros do modelo
Validação	Utilizado para avaliar o modelo durante o treinamento
Teste	Utilizado para avaliar o resultado final

No exemplo das letras A, B e C, a estrutura de diretórios pode ser organizada da seguinte maneira:

dataset/

├── train/

```
| |─ A/  
| |─ B/  
| |─ C/  
|─ val/  
| |─ A/  
| |─ B/  
| |─ C/  
|─ test/  
    |─ A/  
    |─ B/  
    |─ C/
```

A divisão por pastas é adequada para tarefas de classificação, pois o próprio nome do diretório pode representar o rótulo da imagem.

Também é necessário adotar nomes padronizados para os arquivos. Um exemplo seria:

A_0001.png

A_0002.png

B_0001.png

C_0001.png

A numeração impede que imagens sejam sobrescritas e facilita o acompanhamento da quantidade de amostras geradas.

10. Pipeline de geração

Com base no conteúdo apresentado, o processo completo pode ser resumido em cinco etapas:

1. Cena-base

Criação e organização dos objetos, materiais, câmera, iluminação e plano de fundo.

2. Definição dos parâmetros

Escolha das propriedades que serão alteradas, como rotação, cor, posição, escala e intensidade da luz.

3. Randomização

Sorteio de novos valores dentro de intervalos previamente definidos.

4. Renderização

Produção da imagem a partir da configuração atual da cena.

5. Armazenamento

Salvamento da imagem na pasta correspondente à sua classe e divisão.

Cena-base → Parâmetros → Randomização → Renderização → Armazenamento

Esse pipeline demonstra que o resultado mais importante não é apenas a imagem produzida, mas o processo reproduzível utilizado para criá-la.

11. Controle de qualidade

A geração automática de muitas imagens não garante, sozinha, a qualidade do dataset. É necessário avaliar se as amostras produzidas realmente são úteis para o treinamento.

Entre os principais pontos que devem ser verificados estão:

Verificação	Objetivo
Enquadramento	Garantir que o objeto não esteja cortado
Visibilidade	Evitar objetos ocultos ou muito pequenos
Contraste	Manter diferença visual entre objeto e fundo
Diversidade	Produzir variações realmente distintas
Balanceamento	Gerar quantidade semelhante para cada classe
Nomenclatura	Evitar arquivos sobrescritos ou mal organizados
Separação dos dados	Impedir repetições entre treino, validação e teste

A inspeção de algumas imagens durante o processo ajuda a detectar problemas antes que toda a geração seja concluída.

12. Diversidade e randomização de domínio

Um dataset sintético precisa apresentar diversidade suficiente para representar diferentes situações.

A geração de muitas imagens praticamente iguais adiciona pouca informação ao conjunto. Portanto, a quantidade de arquivos deve estar acompanhada de variações relevantes.

Além da rotação e da cor apresentadas no curso, o processo pode ser ampliado para modificar:

- posição do objeto;
- escala;
- distância da câmera;
- ângulo da câmera;
- intensidade das luzes;
- posição das luzes;
- materiais;
- textura do fundo;
- nível de desfoque;
- presença de sombras;
- oclusões parciais.

Essa abordagem está relacionada à randomização de domínio, técnica utilizada para aumentar a variedade das imagens sintéticas e diminuir a dependência do modelo em relação às características específicas da simulação.

Entretanto, a randomização deve ser controlada. Valores muito extremos podem produzir imagens irreais, objetos cortados ou configurações que não representam situações úteis.

13. Da renderização ao dado de treinamento

Uma coleção de imagens renderizadas ainda não constitui necessariamente um dataset completo.

Para ser utilizada em visão computacional, cada imagem precisa estar relacionada a um rótulo ou anotação. No exemplo apresentado, o objetivo é a classificação das letras A, B e C, sendo possível utilizar o nome das pastas como indicação da classe.

Para outras tarefas, seriam necessárias anotações diferentes:

- **Classificação:** classe do objeto;
- **Detecção:** caixas delimitadoras;
- **Segmentação:** máscaras de pixels;
- **Estimativa de profundidade:** mapas de profundidade;
- **Estimativa de pose:** posição e orientação dos objetos.

Uma das vantagens do Blender é que essas informações já existem na cena tridimensional e podem ser extraídas automaticamente. Assim, a imagem e sua anotação podem ser geradas na mesma execução, mantendo correspondência entre os dados.

14. Principais conhecimentos adquiridos

A realização da atividade permitiu compreender que a criação de um dataset sintético depende de duas etapas complementares: a preparação visual da cena e sua automação.

Na primeira etapa, foi possível observar a importância da modelagem, dos materiais, da iluminação, da câmera e das configurações de renderização. Esses componentes determinam a qualidade visual das imagens e precisam ser organizados antes da execução do script.

Na segunda etapa, o Python permitiu transformar uma cena estática em um gerador de dados. A utilização do módulo `bpy`, de valores aleatórios e de laços de repetição possibilitou modificar os objetos e produzir várias imagens automaticamente.

Outro aprendizado importante foi compreender que automatizar não significa apenas gerar uma grande quantidade de arquivos. É necessário definir intervalos adequados, manter a

organização das classes, controlar a visibilidade dos objetos e verificar a qualidade das imagens.

Também foi possível entender a diferença entre uma renderização isolada e um dataset. Enquanto uma imagem depende principalmente de uma boa composição visual, um dataset exige repetição, diversidade, rotulagem, padronização e separação entre treino, validação e teste.

15. Trabalho prático: dataset de cubo, esfera e cone

Como aplicação prática dos conhecimentos apresentados nos vídeos, foi desenvolvido um dataset sintético composto por três classes: cubo, esfera e cone. A escolha desses objetos ocorreu devido às diferenças existentes entre suas formas geométricas, permitindo que cada classe pudesse ser facilmente reconhecida mesmo após alterações de orientação, cor e escala.

Inicialmente, foi criada uma cena contendo um plano para representar o chão, uma superfície de fundo, três fontes de iluminação e uma câmera posicionada de maneira a manter os objetos centralizados. Em seguida, foram adicionados os objetos correspondentes às classes do dataset. Cada elemento recebeu um material próprio, configurado por meio do nó Principled BSDF.

A geração das imagens foi automatizada utilizando um script em Python e o módulo bpy. O código foi responsável por selecionar a classe atual, ocultar os demais objetos, modificar propriedades visuais e executar a renderização. A cada repetição, foram aplicadas variações controladas de rotação, posição, escala, cor e rugosidade.

O controle de visibilidade garantiu que apenas um objeto aparecesse em cada imagem. Dessa forma, durante a geração das amostras do cubo, a esfera e o cone permaneceram ocultos na renderização. O mesmo procedimento foi repetido para as outras classes.

Antes da execução completa, foi realizado um teste reduzido com três imagens de treino, duas de validação e uma de teste para cada classe. Esse teste produziu 18 imagens e permitiu verificar o funcionamento do código, a criação das pastas, o enquadramento da câmera e o salvamento dos arquivos.

Após essa validação, foram geradas 30 imagens de treino, 10 de validação e 5 de teste para cada classe. O dataset final apresentou 135 imagens, sendo 90 destinadas ao treinamento, 30 à validação e 15 ao teste.

As imagens foram armazenadas em formato PNG e resolução de 224×224 pixels. A estrutura das pastas foi organizada inicialmente pelas divisões train, val e test e, dentro de cada uma, pelas classes cubo, esfera e cone.

[Link para o repositório com a parte prática.](#)

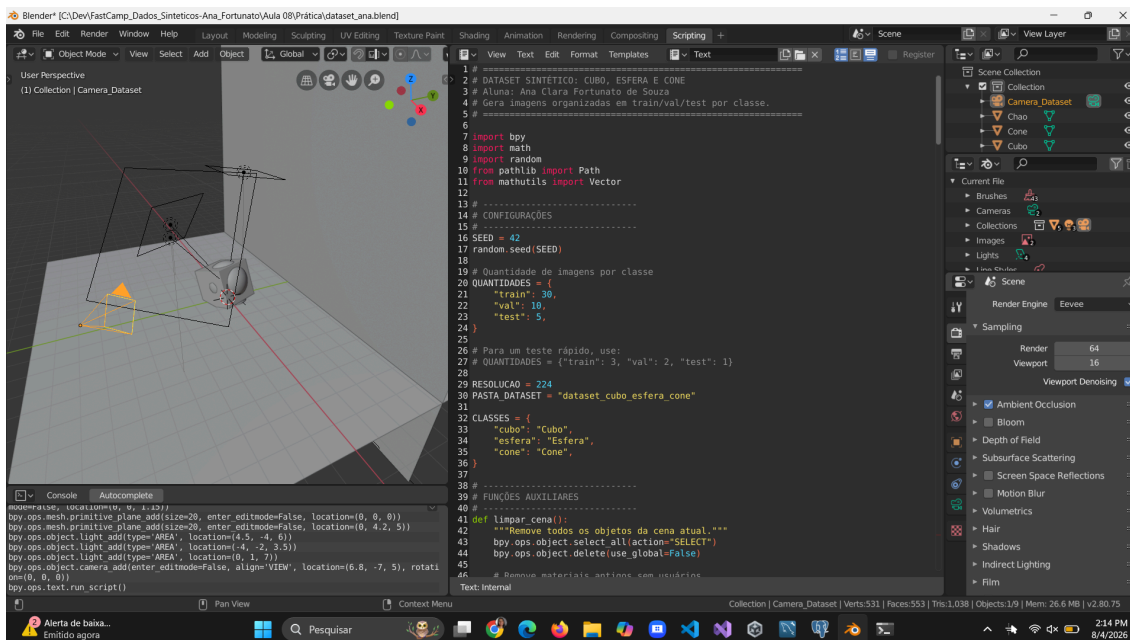


Figura 2 – Ambiente de desenvolvimento no Blender com a cena composta por cubo, esfera e cone, câmera, iluminação e o script em Python utilizado para automatizar a geração do dataset sintético.

Fonte: elaboração própria.

15.1 Conhecimentos adquiridos

A prática permitiu compreender como uma cena tridimensional pode ser transformada em um sistema automatizado de geração de dados. A configuração dos objetos, dos materiais, da câmera e da iluminação constituiu a base visual do dataset, enquanto o script em Python foi responsável pela repetição e pela diversidade das imagens.

Também foi possível compreender a importância da utilização de variações controladas. Alterações muito grandes poderiam posicionar os objetos fora do enquadramento ou torná-los pouco reconhecíveis. Por isso, os intervalos de rotação, posição e escala foram limitados.

Outro aprendizado relevante foi o controle da visibilidade dos objetos durante a renderização. Como o dataset foi elaborado para classificação, cada imagem precisava conter apenas uma das três classes. O script ocultou automaticamente os objetos que não pertenciam à classe atual.

A atividade também demonstrou a importância de executar um teste reduzido antes da geração completa. Dessa forma, possíveis problemas de câmera, iluminação, caminhos de saída ou nomenclatura puderam ser identificados sem a necessidade de renderizar todas as imagens novamente.

Dificuldades

As principais dificuldades estavam relacionadas à organização dos objetos da cena e à adaptação inicial ao ambiente de programação do Blender.

Foi necessário compreender como acessar os objetos e os materiais pelo nome, além de verificar se a câmera e o caminho de saída estavam corretamente definidos. Pequenos erros nessas configurações poderiam impedir a renderização ou salvar as imagens em locais incorretos.

Outro cuidado importante foi limitar as alterações aleatórias para manter os objetos visíveis e corretamente enquadrados. A realização de testes com poucas imagens mostrou-se importante antes da execução de uma sequência maior.

Conclusões

Os dois primeiros vídeos do curso demonstraram de forma prática como o Blender pode ser utilizado para criar um dataset sintético.

A preparação da cena estabelece a base visual do processo. Os objetos, materiais, luzes, câmera e plano de fundo precisam estar corretamente configurados para que as imagens apresentem boa qualidade e mantenham os elementos reconhecíveis.

Em seguida, a utilização do Python permite automatizar as alterações e renderizações. A randomização de rotação e cor, combinada com os laços de repetição, transforma uma cena simples em um gerador capaz de produzir um conjunto variado e organizado.

A atividade também mostrou que a qualidade de um dataset não depende somente da quantidade de imagens. É necessário garantir diversidade, equilíbrio entre as classes, nomenclatura consistente, enquadramento adequado e separação correta entre treino, validação e teste.

Dessa forma, o Blender deixa de ser apenas uma ferramenta de modelagem e renderização e passa a funcionar como uma plataforma programável para a geração de dados destinados a aplicações de inteligência artificial e visão computacional.

Referências

IMMERSIVE LIMIT. **3D Rendered Datasets in Blender for Beginners, Part 1 [Free Course]**. YouTube, 2020. Disponível em: <https://www.youtube.com/watch?v=E1PqpfG5kSo>. Acesso em: 4 ago. 2026.

IMMERSIVE LIMIT. **3D Rendered Datasets in Blender for Beginners, Part 2 [Free Course]**. YouTube, 2020. Disponível em: https://www.youtube.com/watch?v=dG9u_dWDuyA. Acesso em: 4 ago. 2026.

BLENDER FOUNDATION. **Blender Python API Documentation**. Disponível em: <https://docs.blender.org/api/current/>. Acesso em: 4 ago. 2026.